

Search Keyword Revenue Pipeline

Operations Runbook

System	AWS Glue + Amazon S3 (analytics-hitdata-s3)
Entry Point	glue_job.py
Trigger	Manual or scheduled via AWS Glue Console / CLI
Owner	ACS Data Engineering Team
Version	2.0
Last Updated	2026-03-13

1. Overview

This runbook covers all operational procedures for the Search Keyword Revenue Pipeline — an AWS Glue PySpark job that ingests Adobe Analytics hit-level data from S3, validates its schema, and produces a ranked report of revenue attributed to external search engine keywords.

The pipeline is idempotent by design: re-submitting the same source file without the `--force` flag produces no duplicate output. Every execution writes its own timestamped log, run summary, and metadata report to S3.

2. Prerequisites

2.1 AWS IAM Permissions

The IAM role attached to the Glue job must include:

Permission	Purpose
s3:GetObject analytics-hitdata-s3/raw/*	Read source hit-level files
s3:PutObject analytics-hitdata-s3/processed/*	Write analysis output tab file
s3:PutObject analytics-hitdata-s3/logs/*	Write pipeline logs and run summaries
s3:PutObject analytics-hitdata-s3/metadata/*	Write schema profiling reports
s3:GetObject + HeadObject + PutObject checksums/*	Idempotency checksum registry
s3:DeleteObject processed/*/_tmp_output_*/*	Remove Spark temporary part files
s3:ListBucket analytics-hitdata-s3	Required for output consolidation paginator calls
logs:CreateLogGroup / PutLogEvents	CloudWatch job logging

2.2 Glue Job Configuration

Setting	Recommended Value
Glue version	Glue 4.0 (Spark 3.3 / Python 3.10)
Worker type	G.1X for development G.2X for production
Number of workers	2 minimum; increase for files > 2 GB
Job timeout	60 minutes
Max retries	1

Setting	Recommended Value
Script location	s3://analytics-hitdata-s3/scripts/glue_job.py
--extra-files	s3://analytics-hitdata-s3/config/metadata_config.json

3. Job Arguments

Argument	Required	Default	Description
--JOB_NAME	Yes	—	Glue-injected job name. Do not set manually.
--input_key	Yes	—	S3 key of the source .tab file. Example: raw/2026/03/12/14-30-00/data.tab
--force	No	false	Pass 'true' to bypass the SHA-256 checksum guard. Accepted: --force or --force true

WARNING

--force should only be used after a confirmed partial failure or during development/testing. Using it in production without cause will overwrite that day's output file.

4. Pipeline Execution Steps

Step	Name	Description
0	Checksum Guard	Streams source file in 8 MB chunks to compute SHA-256. Checks checksums/ registry. Skips all remaining s
1	Read Source	Reads the tab-delimited file from S3 with Spark CSV reader (sep=tab, quote, escape, multiLine=true, UTF-8)
2	Metadata Profiling	Validates columns against metadata_config.json. Two distributed aggregation passes: min/max/null counts,
3	Business Transform	Extracts search engine domain + keyword from referrer. Identifies purchases (event_list contains '1'). Sums
4	Write Output	Coalesces to 1 partition, writes to temp prefix, copies part file to processed/YYYY/MM/DD/YYYY-MM-DD_S
5	Register Checksum	Writes checksums/<sha256>.json with output key, metadata key, and first_processed_at_utc. Prevents futur
6	Write Run Summary	Writes logs/YYYY/MM/DD/run-summary-<ts>.json with status, record counts, output key, checksum.
7	Flush Pipeline Log	Writes logs/YYYY/MM/DD/pipeline-<ts>.log. Also available in CloudWatch. Error handler writes FAILED sum

5. S3 Output Structure

All artifacts use a date-partitioned path (YYYY/MM/DD) from the UTC run time. Multiple runs per day each produce separate timestamped files and never overwrite each other.

S3 Path	Description
processed/YYYY/MM/DD/YYYY-MM-DD_SearchKeywordRevenuePipeline/output.tab	Table of processed output. Search Engine Domain, Search Keyword, Revenue. Sorted descending by Revenue.
logs/YYYY/MM/DD/run-summary-YYYY-MM-DD-HH-MM-SS.json	Run summary: S3 key, checksum, record counts, output key, timestamp.
logs/YYYY/MM/DD/pipeline-YYYY-MM-DD-HH-MM-SS.log	Timestamped pipeline log. Mirrors CloudWatch output.
metadata/YYYY/MM/DD/metadata-log-YYYY-MM-DD-HH-MM-SS.json	DDL history, schema mismatch details, distinct stats, schema mismatch details, row count.
checksums/<sha256>.json	Idempotency record: file hash → output key + metadata key + first_processed_at_utc.

NOTE

The output .tab file uses only the date in its name. Re-running with --force on the same calendar day overwrites that day's output.

All log, summary, and metadata files always include a full HH-MM-SS timestamp and are never overwritten.

6. Running the Job

6.1 Via AWS Glue Console

1. Open AWS Glue Console and navigate to Jobs.
2. Select SearchKeywordRevenuePipeline and click Run with parameters.
3. Add job parameters:

Key	Value
--input_key	raw/2026/03/12/14-30-00/data.tab (replace with actual S3 key)
--force	true (optional — omit for normal runs)

4. Click Run job and monitor progress in the Glue Console Run tab.

6.2 Via AWS CLI

Standard run:

```
aws glue start-job-run \
  --job-name SearchKeywordRevenuePipeline \
  --arguments '{"--input_key":"raw/2026/03/12/14-30-00/data.tab"}'
```

Force reprocess:

```
aws glue start-job-run \  
--job-name SearchKeywordRevenuePipeline \  
--arguments '{"--input_key":"raw/.../data.tab","--force":"true"}'
```

7. Monitoring and Verification

7.1 Check Run Summary

```
aws s3 cp s3://analytics-hitdata-s3/logs/YYYY/MM/DD/run-summary-.json -
```

Success:

```
{ "status": "SUCCESS", "records_processed": 21, "output_rows": 3, ... }
```

Skipped:

```
{ "status": "SKIPPED_NO_CHANGE", "hint": "Pass --force true to reprocess..." }
```

7.2 Verify Output File

```
aws s3 ls s3://analytics-hitdata-s3/processed/YYYY/MM/DD/
```

Preview:

```
aws s3 cp s3://.../.../YYYY-MM-DD_SearchKeywordPerformance.tab - | head -5
```

Expected format (tab-delimited):

```
Search Engine Domain Search Keyword Revenue
google.com ipod 540.00
bing.com zune 250.00
```

7.3 View Pipeline Log

```
aws s3 cp s3://analytics-hitdata-s3/logs/YYYY/MM/DD/pipeline-.log -
```

Also available in CloudWatch under: /aws-glue/jobs/output

8. Troubleshooting

Symptom	Likely Cause	Resolution
processed/ empty after run	Checksum match — job skipped	Check run summary. Delete checksums/<sha256>.json or pass --force true.
status: SKIPPED_NO_CHANGE	Same file was processed before	Normal for unchanged files. Use --force true or supply a new source file.
FAILED: S3 source file not found	Wrong --input_key value	Verify: <code>aws s3 ls s3://analytics-hitdata-s3/<input_key></code>
No part file found under temp	Spark write failed silently	Check CloudWatch logs. Verify IAM write permissions on processed/ prefix.
WARNING: Schema mismatch	Schema differs from metadata_catalog	Review metadata report. Update config if schema change is intentional.
Output has 0 rows	No purchases with search referrer	Verify <input_key> contains '1' and referrer is a known search engine domain.
Executor OOM / out of memory	coalesce(1) bottleneck for large file	Upgrade to G.2X workers, increase worker count, or remove coalesce(1) for

9. Recovering from Failures

9.1 Failure Before Output Written (Steps 0–3)

No output or checksum was written. Re-run the job normally — the pipeline will proceed from scratch.

9.2 Failure After Output Written (Steps 5–7)

Output may exist but checksum may not be registered. Re-run normally. If checksum is not registered the pipeline reprocesses (safe). If it is, use --force true.

9.3 Intentional Reprocessing

- 1. Option A: Pass --force true — pipeline reprocesses and overwrites that day's output.
- 2. Option B: Delete the checksum entry and re-run normally:

```
aws s3 rm s3://analytics-hitdata-s3/checksums/.json
```

TIP

Always check the run summary status field before deciding to re-run.
SKIPPED_NO_CHANGE is expected and correct for unchanged files — it is not a failure.

10. Scalability Notes

Component	Scales?	Notes
SHA-256 checksum	Yes — linear	boto3 streaming in 8 MB chunks. Constant memory at any file size.
Spark CSV read	Yes — linear	Fully distributed. Scales with workers and file partitions.
Metadata profiling	Yes — constant	Two agg passes returning a single row. No in-driver data collection.
Business transform	Yes — linear	All joins, window functions, and aggregations are distributed Spark. No Python UDFs.
coalesce(1) write	NO — bottleneck	Funnels all data through one executor. Safe up to ~2 GB. For 10 GB+: remove coalesce(1)